QUINTAURIS

De la Nube a la FPGA: Introducción Práctica a RISC-V con Quintauris

Introduction práctica a RISC-V con Quintauris

Open IP · Repos · Toolchain

quintauris.com

Agenda

2 Semanas – 8 sesiones

SEMANA 1

LUN 7
17:00–19:00

1. Fundamentos de RISC-V: de la arquitectura al ecosistema

Quintauris · Ángel Berrio

MAR 8
17:00–19:00

2. Presentación de Quintauris: visión, modelo de negocio y productos

Quintauris · Ángel Berrio

MIÉ 9
17:00–19:00

3. Explorando la plataforma Quintauris, Tools and Infra — incluyendo Testing, Benchmarking y QNTAF (I)

Quintauris · Enrique Pallares

JUE 10
17:00–19:00

4. *Introducción practica a RISC-V con Quintauris*

Quintauris · Enrique Pallares

SEMANA 2

LUN 14
16:30–19:30

5. IP hardware RISC-V open source: introducción, licencias y repositorios

Quintauris · Ignacio Ruiz

MAR 15
16:30–19:30

6. Parte I: De la nube a la FPGA — flujo de despliegue de una IP en la FPGA

UGR & Quintauris · Jose Luis Martínez, Ignacio Ruiz / UGR

MIÉ 16
16:30–19:30

7. Parte II: De la nube a la FPGA — flujo de despliegue de una IP en la FPGA

UGR & Quintauris · Jose Luis Martínez, Ignacio Ruiz / UGR

JUE 17
16:00–20:00

8. Hackathon final: integración de conceptos y solución de un reto práctico

Quintauris · Jose Luis Martínez, con mentores de UGR

Session 4

- *Introducción practica a RISC-V con Quintauris*

THE POINT OF TODAY

A modern processor is something you can clone, read and build — not something you buy blind.

OUTCOME 01

A real core on your disk

You can navigate an industrial IP repository and say what each folder is for.

OUTCOME 02

A toolchain that answers you

You compile C for RISC-V and read the instructions your choices produced.

OUTCOME 03

A map from RTL to board

You know every step between source files and a bitstream on an FPGA.

Agenda

00	Welcome & environment check	Everyone on the same starting line
01	The open IP ecosystem	Who makes cores, and why
02	Choosing the right IP	Five questions + live vote
03	Lab 1 — Clone & explore two repos	NEORV32 and CVA6, side by side
04	Lab 2 — Your RISC-V toolchain	Install, compile, disassemble
05	Tooling & deployment intro	Vivado, openFPGALoader, Docker

On Windows? You need WSL first

Native Windows has no make, no grep , no shell scripting — and every hardware flow assumes all three. WSL2 gives you real Linux inside Windows.

```
# PowerShell as administrator, once
> wsl --install -d Ubuntu
# reboot, then create your user

# from now on: the Ubuntu terminal
$ sudo apt update
$ sudo apt install -y build-essential \
git python3 python3-pip curl

# macOS: shell tools come from Xcode
$ xcode-select --install
```

Clone inside Linux, not /mnt/c

Work in your Linux home. Crossing filesystems is up to 10× slower and breaks permissions and symlinks.

Edit in Windows, build in Linux

VS Code + the Remote—WSL extension: your editor stays on Windows, every command runs on Ubuntu.

JTAG over USB comes later

Passing a programmer into WSL needs usbipd-win . Nothing today depends on it — session 5 does.

Run this now. Paste the output in the chat.

```
# one line, four answers
git --version && python3 --version \
&& make --version | head -1 && df -h .

# optional, but nice to have today
docker --version
code --version
```

✓ **git ≥ 2.30**
submodules and shallow clones

✓ **python3 ≥ 3.8 + make**
every build flow leans on both

✓ **15 GB free disk**
CVA6 with submodules is over 1 GB

! **Vivado is not needed today**
We only look at its flow. Install before session 5.

01

The open IP ecosystem

The ISA is free. The interesting work — and the competition — moved to the implementations.

"IP" means someone already solved this in hardware

A core, a bus, a UART, a memory controller — a reusable, parameterised design you integrate instead of writing. Three ways to get one:

License it

Vendor core, support contract, NDA, per-chip royalties. Predictable, closed, expensive.

$$$ · weeks of legal

Build it

Full control, full cost. Verification alone dwarfs the design effort.

team-years

Clone it

Open RTL under a permissive licence. You read every line — and you own integration and verification.

git clone · today

Open does not mean free of work. Licence terms, verification evidence and system integration remain your responsibility that gap is precisely what Quintauris exists to close for industrial products.

Five layers — and where today sits

Specification	The ISA, profiles and extensions. Free to read, ratified by committee.	riscv/riscv-isa-manual
Cores ← today	CPU RTL: NEORV32, CV32E40P, Ibex, CVA6, VexRiscv, Rocket.	openhwgroup · pulp-platform
SoC platforms	Core + bus + memory + peripherals + debug. This is what boots.	pulpissimo · cheshire · litex
Verification	Testbenches, random instruction generators, compliance suites. The real cost.	core-v-verif · riscv-dv
Tooling ← today	Compilers, simulators, synthesis, programmers, containers.	gcc · verilator · yosys

OpenHW Group

github.com/openhwgroup

A non-profit consortium doing collaborative, **verified** core development in the open — with the same rigour a semiconductor company applies internally.

- → Industrial-grade, open-source cores — the CORE-V family
- → Verification is a first-class deliverable, not an afterthought
- → Members include silicon vendors, universities and EDA companies

CV32E40P

openhwgroup/cv32e40p

RV32IMC 4-stage in-order MCU core, ex-RI5CY. The reference "my first industrial core".

CVA6

openhwgroup/cva6

Application-class, Linux-capable. Next slide.

core-v-verif

openhwgroup/core-v-verif

The verification environment. Open this before you trust any core.

PULP Platform

github.com/pulp-platform

ETH Zürich & University of Bologna. Modular cores *and* complete subsystems for IoT, ML and embedded — the academic engine room of open RISC-V.

- → Origin of RI5CY (now CV32E40P) and Ariane (now CVA6)
- → Interconnect, DMA, caches and accelerators you can reuse standalone
- → Dependency management via Bender — you will meet Bender.yml in Lab 1

PULPissimo

pulp-platform/pulpissimo

Single-core microcontroller system: core, memory, peripherals, debug. A real SoC to study.

Cheshire

pulp-platform/cheshire

Linux-capable host platform built around CVA6. Where cores become computers.

CV32E40X / CVA6

now maintained at openhwgroup

Research → industrialisation. Follow the rename history; the papers still say RI5CY and Ariane.

CVA6 (ex-Ariane)

github.com/openhwgroup/cva6

A 64-bit application-class core in SystemVerilog. Six-stage, in-order, with MMU, caches and full privileged architecture — it boots Linux.

```
# what "Linux-capable" really costs you
MMU + caches + PLIC + CLINT
+ DRAM controller + bootloader
+ device tree + OpenSBI
= a system, not a core
```

Configurable width

CV32A6 (32-bit) and CV64A6 (64-bit) from the same source tree.

Real FPGA targets

Genesys 2, Nexys Video, VCU118. Needs a mid/large FPGA — tens of thousands of LUTs.

Used in silicon

Taped out in academic and commercial chips — the credibility argument for open IP.

Start this clone now — recursive submodules take a while. Details in Lab 1.

VexRiscv

github.com/SpinalHDL/VexRiscv

Highly configurable core written in SpinalHDL (Scala). You do not edit RTL — you configure a plugin pipeline and *generate* Verilog.

```
// plugins in, pipeline out
new IBusCachedPlugin(...)
new DBusCachedPlugin(...)
new MulDivIterativePlugin(...)
→ VexRiscv.v
```

Lightweight & fast to synthesize

From a few hundred LUTs (minimal RV32I) up to a Linux-capable configuration.

FPGA-first mindset

Excellent LUT/MHz numbers on Xilinx, Lattice and Intel parts.

Pairs with LiteX

enjoy-digital/litex

Python SoC builder: pick a core, a board, peripherals — get a bitstream.

Cost of entry: a JVM and Scala toolchain. Generated code is not meant to be read by humans.

NEORV32

github.com/stnolting/neorv32

A customizable microcontroller-like SoC built around the NEORV32 RISC-V CPU, written in **platform-independent VHDL** .

Designed as an auxiliary controller inside larger SoCs — or as a tiny custom microcontroller that even fits a Lattice iCE40 UltraPlus.

> It works out of the box, and it targets beginners *and* advanced users. That is rare, and it is why we build on it.

HDL
VHDL
No vendor primitives, no IP cores

ISA
rv32 i/e/m/c/b...
Plus a long list of Z* extensions

System
Full SoC
UART, SPI, TWI, GPIO, timers, WDT

Debug
On-chip JTAG
GDB over the standard debug module

Software
Bootloader + BSP
Upload firmware over UART, no re-synthesis

Docs
A real datasheet
Read it before touching the RTL

Three more names you will hear

Ibex + OpenTitan

lowRISC/ibex · lowRISC/opentitan

Small, security-hardened RV32 core inside the first open silicon root of trust. Exemplary documentation and DV practice.

Rocket & Chipyard

ucb-bar/chipyard

The Berkeley/Chisel world: generators, out-of-order BOOM cores, full SoC agile design flows.

RISC-V International

riscv.org/technical/specifications

The specs themselves, plus the profiles that say which extensions a real product must implement.

Five cores, one table

CORE	SOURCE	ISA	LINUX	SIZE	REACH FOR IT WHEN…
NEORV32	VHDL	rv32imc_b…	no	~2–5k LUT	you want a working MCU on any FPGA today
CV32E40P	SystemVerilog	rv32imc(f)	no	~4–8k LUT	you need verification evidence for a product
Ibex	SystemVerilog	rv32imc	no	~2–6k LUT	security and lockstep matter more than speed
VexRiscv	SpinalHDL	rv32i…imac	opt.	0.5–10k LUT	you want to dial area vs. performance freely
CVA6	SystemVerilog	rv64gc / rv32	yes	40k+ LUT	you need an operating system, not a controller

Sizes are order-of-magnitude for a mid-range 7-series FPGA and depend heavily on configuration. Always re-measure for your own build.

Five questions, in this order

Q1	Does it have to run an OS?	Yes → CVA6, Rocket, Linux-VexRiscv. No → stay MCU-class and save 10× the area.
Q2	What language does your team read?	VHDL → NEORV32. SystemVerilog → Ibex, CV32E40P, CVA6. Scala/Chisel → VexRiscv, Rocket.
Q3	How many LUTs and BRAMs do you have?	Your board sets the ceiling.
Q4	Do you need the whole system or just the CPU?	Need UART, timers, debug on day one? Take an SoC (NEORV32, PULPissimo, LiteX), not a bare core.
Q5	Who maintains it, under which licence, with what proof?	Last commit, open issues, CI status, releases, Solderpad/Apache terms, and a verification report.

Four traps that cost weeks

"A core is a computer"

A CPU with no bus, memory, reset strategy or clocking does nothing on a board. Budget for the system around it.

"It compiles, so it works"

Synthesis success says nothing about correctness. Simulation and compliance testing do. Session 4.

"Open means no licence"

Solderpad, Apache-2.0, MIT, CERN-OHL all impose different attribution and patent terms. Read the LICENSE file first.

"Custom extensions are free"

Add an instruction and you now maintain a compiler, an assembler, a simulator model and a testbench for it.

02

Cloning & exploring IP repositories

Terminals open. From here on, you type along with me.

Every IP repo has the same skeleton

```
neorv32/
├── rtl/ the hardware
│   ├── core/ CPU + SoC
│   └── system_integration/
├── sw/ C runtime, examples
├── sim/ simulation scripts
├── setups/ board projects
├── docs/ the datasheet
└── LICENSE read me first
```

rtl/ or src/

Synthesisable source. Find the *_top file — it is your contract with the outside world.

tb/ or sim/ or dv/

Non-synthesisable testbenches. Never mix these into synthesis.

*.core / Bender.yml / *.f

Dependency and filelist manifests — FuseSoC, Bender, or a plain list. The hardware package.json.

.gitmodules

Half the design may live elsewhere. Forget --recursive and you get empty folders and cryptic errors.

Clone a microcontroller. Read its knobs.

```
$ git clone --depth 1 \
https://github.com/stnolting/neorv32.git
$ cd neorv32 && ls

# how big is a whole microcontroller?
$ find rtl -name '*.vhd' | wc -l
$ wc -l rtl/core/neorv32_cpu.vhd

# the knobs: every configurable feature
$ grep -c ':=' rtl/core/neorv32_top.vhd
$ grep -n 'RISCV_ISA' rtl/core/neorv32_top.vhd \
| head -20

# who else is in here?
$ ls setups/ && ls sw/example/
```

What you should see

Dozens of VHDL files, a CPU of a few thousand lines, and a top level with a long list of generics — **that list is the product.**

Three answers to find

1. Which ISA extensions can you switch on?
2. How much internal memory can you configure?
3. Which peripheral would you delete first?

Then open docs/ and skim the datasheet's memory map. Never touch RTL you have not read the datasheet for.

Now clone something that boots Linux

```
git clone https://github.com/openhwgroup/cva6.git
cd cva6

# the step everyone forgets
git submodule update --init --recursive
# several minutes. keep talking.

du -sh .git . && cat .gitmodules | grep path
ls core/

# find the pipeline in the file names
ls core/frontend core/cache_subsystem
grep -rn 'CVA6Cfg' core/cva6.sv | head -5
```

The size lesson

Over a gigabyte, dozens of submodules, a configuration package instead of a generic list. Application-class IP is a **federation of repos.**

Map it to session 2

frontend → fetch & branch prediction issue_stage → scoreboard cache_subsystem → I$ / D$ mmu_sv39 → virtual memory

If the clone stalls: --depth 1 on submodules or grab the release tarball. Do not fight a hotel Wi-Fi.

Judge a repo in 90 seconds

SIGNAL 01

Commits & releases

Steady commits and tagged releases beat a burst of stars. Check the last 6 months, not the total.

SIGNAL 02

CI that runs simulation

A green badge that only lints is decoration. Look for regression runs in .github/workflows.

SIGNAL 03

Issues answered

Open an issue thread. A maintainer replying within days is worth more than any README.

SIGNAL 04

Docs that match the code

A datasheet with a memory map and a version history. If docs lag the RTL, you will debug the difference.

SIGNAL 05

A named licence

Solderpad 2.0, Apache-2.0, MIT, CERN-OHL. "No licence file" means all rights reserved.

SIGNAL 06

Someone has taped it out

Silicon or a documented FPGA bring-up is the strongest evidence an IP block is real.

03

Setting up your RISC-V toolchain

You have hardware source. Now you need software that speaks its dialect.

A toolchain is five programs and two flags

gcc	C/C++ → RISC-V machine code
as / ld	assemble and link — the linker script sets your memory map
objdump	read back the instructions — your microscope
objcopy	ELF → bin/hex for a bootloader or a memory initialisation file
gdb	debug over JTAG once the core is on the board

```
# the name tells you the target
riscv32-unknown-elf-gcc
arch — vendor — bare metal (vs. linux-gnu)
```

-march=rv32imc

Which instructions the compiler may emit. Must be a subset of what your core implements — ask for m on a core without a multiplier and you get an illegal-instruction trap.

-mabi=ilp32

How arguments and floats are passed. ilp32 / ilp32f / lp64d . Mismatch it and the linker refuses your libraries.

The one rule: the flags describe the *hardware you built* , not the software you wish you had.

Three ways in — pick A today

A Prebuilt binaries

Download, unpack, add to PATH. Minutes, not hours.

```
# xPack GNU RISC-V
xpack-dev-tools/
riscv-none-elf-gcc-xpack
$ riscv-none-elf-gcc --version
```

✓ Recommended for the course

B Build from source

The only route if you add custom instructions.

```
$ git clone riscv-collab/
riscv-gnu-toolchain
$ ./configure \
--prefix=/opt/riscv \
--with-arch=rv32imc \
--with-abi=ilp32
$ make -j$(nproc)
```

~1 h · ~20 GB disk

C OSS CAD Suite

One tarball: gcc, Verilator, Yosys, nextpnr, GTKWave, openFPGALoader.

```
# YosysHQ
oss-cad-suite-build
$ source oss-cad-suite/
environment
```

Best value if you also want simulation today

xPack GNU RISC-V GCC en /opt

1 · Descarga y descomprime

```
cd /tmp
curl -L -o xpack-riscv.tar.gz https://github.com/xpack-dev-tools/riscv-none-elf-gcc-xpack/releases/download/v15.2.0-1/xpack-riscv-none-elf-gcc-15.2.0-1-linux-x64.tar.gz
sudo mkdir -p /opt/xpack
sudo tar -xzf xpack-riscv.tar.gz -C /opt/xpack
rm xpack-riscv.tar.gz
```

2 · PATH para todos los usuarios

```
echo 'export PATH="/opt/xpack/xpack-riscv-none-elf-gcc-15.2.0-1/bin:$PATH"' | sudo tee /etc/profile.d/xpack-riscv.sh
sudo chmod +x /etc/profile.d/xpack-riscv.sh
source /etc/profile.d/xpack-riscv.sh
```

3 · Verifica

```
riscv-none-elf-gcc --version
```

Salida esperada

riscv-none-elf-gcc (xPack GNU RISC-V Embedded GCC, 32-bit) 15.2.0

La versión cambia

Comprueba con ls /opt/xpack que la carpeta se llama igual que la ruta del PATH.

Sin sudo

Descomprime en ~/opt/xpack y exporta el PATH en tu ~/.bashrc: funciona igual.

En Windows se instala **dentro de Ubuntu/WSL .** En macOS o ARM cambia el sufijo del tarball: darwin-x64, darwin-arm64 , linux-arm64.

Compile the same C twice. Watch the ISA appear.

```
$ printf 'int f(int a,int b){return a*b;}' > f.c

# 1. with the M extension
$ riscv-none-elf-gcc -O2 -march=rv32imc \
-mabi=ilp32 -c f.c -o f_m.o
$ riscv-none-elf-objdump -d f_m.o | tail -6

# 2. without it
$ riscv-none-elf-gcc -O2 -march=rv32i_zicsr \
-mabi=ilp32 -c f.c -o f_i.o
$ riscv-none-elf-objdump -d f_i.o | tail -6

# 3. size the difference
$ riscv-none-elf-size f_m.o f_i.o
```

Expected: with M

mul a0,a0,a1 ret

Expected: without M

jal __mulsi3 a software multiply routine

The lesson

One hardware multiplier — a few hundred LUTs — replaces a subroutine and dozens of cycles.

Reading an ISA string

```
rv32 i mc _zicsr_zifencei
```

rv32 / rv64

Register width. Changes your ABI and your pointer size.

i / e

Base integer set. e = 16 registers, for the smallest cores.

m / a / f / d

Multiply, atomics, single- and double-precision float. Each one costs area.

c

Compressed 16-bit encodings. Typically, 25–30 % smaller code — almost always worth it.

g

Shorthand for imafd_zicsr_zifencei. rv64gc is the Linux baseline.

b

Bit manipulation. Cheap, and a big win in embedded code.

z*

Fine-grained extensions, underscore-separated. CSR access and fence.i now live here.

Classic bug: a tutorial from 2019 says rv32i and modern gcc errors on CSR access. Add _zicsr.

04

Tooling & deployment intro

From source files to a blinking LED — the whole map, plus the exact commands.

Six steps from source to silicon behaviour

01

RTL + config

Pick your generics. Decide the ISA you will support.

minutes

02

Simulate

Verilator or GHDL + GTKWave. Catch bugs here, not on the board.

seconds–min

03

Constrain

Clock period and pin assignments — the .xdc . Part of the design.

once per board

04

Synthesise

Vivado or Yosys. RTL → LUTs, FFs, BRAMs, DSPs.

min–hours

05

Place, route, timing

Where timing fails. Read the report before you read the errors.

min–hours

06

Bitstream → board

openFPGALoader over JTAG, then firmware over UART.

seconds

Firmware changes do **not** require steps 04–05. With a bootloader you rebuild software in seconds — that is why NEORV32 ships one.

Bitstream generation: script it, don't click it

```
# build.tcl — the whole flow, reviewable
read_vhdl -library neorv32 [glob rtl/core/*.vhd]
read_xdc constraints/arty_a7.xdc

synth_design -top neorv32_test_setup_bootloader \
-part xc7a35ticsg324-1L
opt_design
place_design
route_design

report_timing_summary -file timing.rpt
report_utilization -file util.rpt
write_bitstream -force neorv32.bit

$ vivado -mode batch -source build.tcl
```

Two reports decide everything

Utilization — did it fit? **Timing** — is worst negative slack positive? If either fails, the bitstream is a lie.

Why non-project mode

One file in git reproduces the build on any machine. GUI projects do not review, diff or run in CI.

The open alternative

yosys → nextpnr → icepack for Lattice parts. Fully open, seconds per build, no 100 GB install.

Part numbers and top-level names differ per board. Copy the ready-made project under setups/ instead of typing them.

openFPGALoader — one command, any board

```
# is the cable there?
$ openFPGALoader --detect

# load into SRAM — gone at power-off
$ openFPGALoader -b arty_a7_35t \
neorv32.bit

# write flash — survives reboot
$ openFPGALoader -b arty_a7_35t -f \
neorv32.bit

# then talk to your core
$ picocom -b 19200 /dev/ttyUSB0
```

Vendor-neutral

AMD/Xilinx, Lattice, Intel/Altera, Gowin, Efinix — hundreds of boards and cables, one CLI.

trabucayre/openFPGALoader

The toolchain moves to the cloud. The board stays on your desk.

```
# same tools, every machine, no installs
$ docker run --rm -it \
-v $PWD:/work -w /work \
hdlc/sim:osvb \
bash -lc 'verilator --version'

# image catalogue & recipes
github.com/hdl/containers

# and the same thing in CI
jobs:
sim:
container: hdlc/sim:osvb
steps: [ run: make -C sim ]
```

Reproducibility

"Works on my machine" is a hardware problem too. Pin the tool versions in an image and the bug is reproducible everywhere.

Elastic compute

Synthesis and regressions are embarrassingly parallel. Rent 64 cores for an hour instead of waiting overnight.

Where it stops

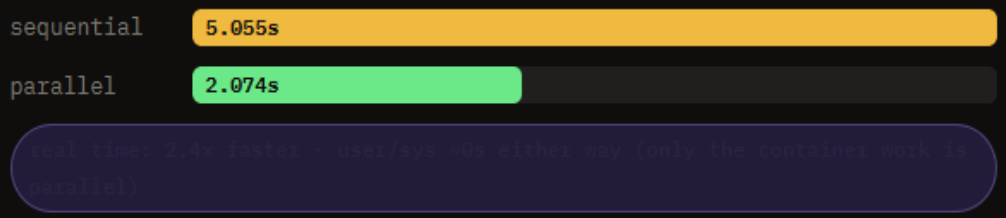
JTAG needs USB. Programming stays local — or on a lab machine with a shared board farm.

05A · DEPLOYMENT — LIVE RESULT

8 containers, run sequential vs parallel — 2.4× faster wall clock.

```
# sequential — one docker run after another
$ time (for i in $(seq 1 8); do
    docker run --rm hdlc/sim:osvb bash -lc 'verilator --version'
  done)
```

```
# parallel — all 8 launched at once, wait for the last one
$ time (for i in $(seq 1 8); do
    docker run --rm hdlc/sim:osvb bash -lc 'verilator --version' &
  done; wait)
```



real is the number that matters

`user` and `sys` stayed near-zero in both runs — they only clock the parent shell, not the work inside each container. `real` is wall-clock, and it's the one that dropped.

independent jobs parallelize almost for free

8 stateless `docker run` calls with no shared state between them collapse toward the slowest single job, not the sum of all eight — the same reason a regression suite scales across rented cores.

the gap is mostly fixed overhead, not real work

Each job here is just `verilator --version` — a few hundred ms of container startup. With real sim/synthesis jobs (minutes each), the same trick saves far more than 2.4×.

where it stops

This proves elastic compute, not board access. JTAG and USB still need a machine with the board physically attached — that part of the pipeline stays local.

measured on WSL2 · Docker 29.1.3 · image: hdlc/sim:osvb

REFERENCE

Everything from today, in one place

Cores & SoCs

github.com/stnolting/neorv32

github.com/openhwgroup

github.com/openhwgroup/cva6

github.com/openhwgroup/cv32e40p

github.com/pulp-platform

github.com/pulp-platform/pulpissimo

github.com/pulp-platform/cheshire

github.com/SpinalHDL/VexRiscv

github.com/lowRISC/ibex

github.com/ucb-bar/chipyard

Software tools

github.com/riscv-collab/riscv-gnu-toolchain

github.com/xpack-dev-tools/riscv-none-elf-gcc-xpack

github.com/YosysHQ/oss-cad-suite-build

github.com/verilator/verilator

github.com/riscv-software-src/riscv-isa-sim

github.com/riscv/riscv-isa-manual

riscv.org/technical/specifications

github.com/olofk/fusesoc

github.com/pulp-platform/bender

Hardware & flow

github.com/trabucayre/openFPGALoader

github.com/hdl/containers

github.com/enjoy-digital/litex

github.com/openhwgroup/core-v-verif

github.com/chipsalliance/riscv-dv

github.com/lowRISC/opentitan

github.com/YosysHQ/nextpnr

quintauris.com/professional-services

You now have a core, a compiler, and the map between them.

01 Cloned 02 Compiled 03 Mapped to a board